

Rahul Chaudhary

Generative AI Engineer | Full Stack Developer
Evidence-Based Resume & Codebase Interview Playbook

Candidate: Rahul Chaudhary · B.E. Electronics & Communication Engineering, CCET

GitHub: github.com/RRNauhwar · **Target roles:** GenAI / AI Engineer, Full Stack, Backend

Prepared as: Senior Interviewer · Hiring Manager · ATS Reviewer · Principal Engineer

Scope: 12 sections — Resume audit, code validation, 900+ questions, project deep-dives, behavioral, HR, coding, company prep, revision & cheat sheet

Grounded in real evidence. Every technical claim in this handbook was checked against your actual source code on GitHub (NyayaSim / Legal-Ai-App, TradeLab-AI, Bookstore-DBMS). Where the resume and the code disagree, this book tells you exactly where — *before an interviewer finds it.*

READ THIS FIRST

How to Use This Handbook

This is not a generic question dump. It was built by reading your resume line by line and then opening your actual repositories to check whether the code backs up the words. That is exactly what a strong interviewer does.

The single most important thing to understand: your resume describes NyayaSim as a “*React, Node.js, Express.js, Supabase, PostgreSQL*” project. Your code tells a richer and slightly different story — there is also a full **Kotlin/Jetpack Compose Android app**, the deployed AI model is **Anthropic Claude** (not OpenAI/Gemini), and the backend persists data in a **JSON flat file**, not PostgreSQL. None of this is bad — but you must be the one who explains it, not the interviewer who discovers it.

How the question cards work

Q. Sample question appears in bold like this.

Why they ask: the hidden signal the interviewer is testing for.

Ideal answer: a concrete, evidence-backed response you can actually defend.

Common mistake: the trap most candidates fall into.

Follow-up: where the interviewer will push next.

Difficulty tags used throughout: **BEGINNER** **INTERMEDIATE** **ADVANCED** **EXPERT / STAFF**

Priority stars mark how likely a question is, given *your specific* resume: almost certain ·
likely · possible.

Golden rule for every answer: explain it simply first (as if to a smart non-engineer), then add the technical depth. Never claim something the code cannot back up. If you are unsure, say “*Here’s what I built, and here’s what I’d do differently now*” — honesty plus growth beats bravado every time.

Table of Contents

1. Resume Audit

Section-by-section review, strengths, weaknesses, red flags, ATS, scores /10

2. Resume Validation vs. Code

Every claim rated: supported / partial / not supported — with file evidence

3. Complete Resume Interview Questions

Questions generated from every line: intro, why-this, challenges, trade-offs

4. Project-by-Project Deep Dive

NyayaSim, Smart Glasses, DealHunt, Telegram News — explanations, architecture, Q banks

5. Skills Verification

Per-skill basic advanced coding scenario questions with strong-answer notes

6. Cross Questions

The skeptical interviewer: relentless follow-up chains until the topic is exhausted

7. Resume Weaknesses

Weak wording, unsupported buzzwords, missing metrics — with stronger rewrites

8. Behavioral Interview

STAR-format answers drawn from your real projects and achievements

9. HR Interview

Tell me about yourself, strengths/weaknesses, goals, salary, relocation

10. Coding Interview

Predicted DSA + SQL + OOP + API questions with complexity and tips

11. Company-Specific Preparation

Google, Amazon, Microsoft, Adobe, product companies, startups

12. Final Revision Guide + Night-Before Cheat Sheet

Top-N lists, one-page summaries, pitfalls, highest-probability questions

A note on completeness: your resume lists four projects but only **NyayaSim** (repo [Legal-Ai-App](#)) is publicly available as source code, alongside two strong repos not on your resume ([TradeLab-AI](#), [Bookstore-Database-Management-System](#)). DealHunt, Smart Glasses, and the AI Telegram News Assistant could not be located as public repositories, so their sections are based on your resume wording plus standard architecture for those stacks — and are clearly labelled as **unverified against code**.

Resume Audit

A section-by-section review the way a hiring manager reads it in the first 30 seconds — what they notice, what helps you, what hurts you, and a score out of 10.

Overall first impression (recruiter's 6-second scan): Clean one-page, ATS-friendly CCET template, strong keyword coverage (GenAI, RAG, Agentic AI, React/Node, Docker). Positioning as “Generative AI Engineer | Full Stack Developer” is current and marketable. The GATE CS qualification while doing an ECE degree is an eye-catching differentiator. Biggest risks: a few high-value buzzwords (RAG, Agentic AI, PostgreSQL) are not fully backed by the visible code, and your strongest engineering artifact — a complete Kotlin Android app — is invisible on the resume.

Overall resume score: 7.4 / 10 — a genuinely good student resume; the gap between claims and evidence is what stands between you and an 9.

1.1 Education 8 / 10

What they notice first: B.E. ECE at CCET, expected 2027, CGPA 7.37, plus strong Class X (91.8%) and XII (82%).

- **Strengths:** Clear table, reputable institute, consistent academic record. The “Expected graduation 2027” is honest and correctly flagged.
- **Weaknesses:** CGPA 7.37 is middling; some product companies filter at 7.5 or 8.0. You cannot change it, so you must out-signal it with projects, GATE, and DSA rank.
- **Red flags:** None. The date range is transparent.
- **ATS:** Real table text, standard heading — parses fine.
- **Improvement:** Consider adding one line of relevant coursework (Data Structures, DBMS, OOP, Computer Networks, OS) — it helps for a CS-adjacent role coming from an ECE degree, and feeds ATS keyword matching.

1.2 Professional Summary 7 / 10

What they notice first: “builds complete AI-powered products end to end... LLM applications, RAG pipelines, and agentic AI workflows.”

- **Strengths:** Tight, role-aligned, keyword-rich, no fluffy “hardworking team player” filler.
- **Weaknesses / red flag:** “RAG pipelines” is not demonstrated in any public repo (no vector store, no embeddings, no retrieval step). This is the single most dangerous word on your resume — see Section 2. “Agentic AI” is also a stretch relative to the code (multi-persona prompting, not autonomous tool-using agents).
- **Improvement:** Either (a) build one genuine RAG feature (embed the Indian legal sections / case corpus and retrieve them before prompting) so the claim becomes true, or (b) soften to “LLM

applications and prompt-orchestration pipelines.” Do not leave an undefendable claim in the very first paragraph.

1.3 Technical Skills 7 / 10

- **Strengths:** Well-grouped (Languages / Web & DB / GenAI / Tools & Embedded), all commonly-screened keywords present.
- **Weaknesses:** Two problems. (1) **Missing skills you actually demonstrate:** Kotlin, Android (Jetpack Compose), Room, Hilt/DI, MVVM, Prisma, Redis, SQL — all present in your repos but absent here. (2) **Skills that over-promise:** “RAG” and “Agentic AI” (see above). Listing a skill invites a deep question; only list what you can survive 4 follow-ups on.
- **Red flag:** “PostgreSQL” next to NyayaSim implies the NyayaSim backend uses Postgres. It does not — it uses a JSON file. Your real Postgres evidence is in the Bookstore and TradeLab-AI repos, which aren’t listed.
- **ATS:** Good. Comma-separated, parseable.
- **Improvement:** Add a “Mobile” or extend “Web & Databases” line to include Kotlin/Android/Room; add Prisma & Redis (backed by TradeLab-AI). Remove or qualify RAG until it’s real.

1.4 Experience — AI Automation Intern, D2 Automation 7 / 10

- **Strengths:** Real internship, relevant stack (n8n + OpenAI/Gemini + Telegram bot + Python), action verbs, clear outcomes.
- **Weaknesses:** Short duration (June–July 2025 2 months) will draw “what did you actually own?” questions. No metrics (how many workflows, how much manual time saved, how many queries handled).
- **Red flag / overlap:** The internship’s “Telegram chatbot” overlaps heavily with your project “AI Telegram News Assistant.” An interviewer may ask whether these are the same work counted twice. Have a crisp distinction ready.
- **Improvement:** Add one quantified bullet, e.g. “Automated N recurring workflows, cutting a ~X-hour manual task to minutes.” Even approximate, honestly-labelled numbers help.

1.5 Projects 7.5 / 10

- **Strengths:** Four varied projects across GenAI, embedded/CV, and full-stack — good breadth for a student. NyayaSim is genuinely impressive and substantiated by real, well-structured code.
- **Weaknesses:** (1) Only one project (NyayaSim) is publicly verifiable; the other three have no locatable public repo, which is risky if an interviewer asks “can you share the code?”. (2) No metrics anywhere (users, latency numbers, accuracy, dataset size). (3) NyayaSim’s bullet claims “PostgreSQL” and “Agentic AI” that the code only partially supports.
- **Ordering:** Correct — NyayaSim first (strongest). But consider swapping DealHunt and Smart Glasses depending on the role: for AI roles keep the two AI-heavy ones up top; for embedded/hardware roles, promote Smart Glasses.
- **Improvement:** Add a GitHub link per project. Add one measurable outcome per project. Feature TradeLab-AI (a very strong full-stack + AI + Prisma/Redis/Docker repo) — it may be more impressive than DealHunt.

1.6 Certifications 8 / 10

- **Strengths:** Relevant and coherent (Python, Full-Stack GenAI, Web Dev, a govt-backed drone bootcamp with a concrete score 27/30). They map directly to the target role.
- **Weaknesses:** Udemy certificates carry limited weight at top product companies; they signal initiative more than mastery. The interviewer will test whether you retained the content, not that you finished it.
- **Improvement:** Keep them, but be ready to demonstrate depth beyond “I did the course” — tie each cert to something you built.

1.7 Achievements 8.5 / 10

- **Strengths:** This is your strongest differentiator block. **GATE CS qualified** (from an ECE background) signals CS fundamentals; **170+ DSA problems, 16th on institute leaderboard** signals coding discipline; a hackathon build signals delivery under pressure.
- **Weaknesses:** “GATE qualified” without a score/rank is weaker than “GATE CS, score X / AIR Y.” “170+ on GeeksforGeeks” is good but LeetCode is the currency most product companies recognize.
- **Improvement:** Add the GATE score/rank if respectable. Consider mirroring some DSA onto LeetCode and citing that count too.

1.8 GitHub 6.5 / 10

- **Strengths:** Active, public, many repos across TypeScript/Python/Java — shows consistent building.
- **Weaknesses / red flags an interviewer will find:** (1) Most repos have **no README, no description, no license**. (2) The repo behind NyayaSim is named [Legal-Ai-App](#), not “NyayaSim,” so a recruiter searching your GitHub for “NyayaSim” finds nothing — a naming mismatch that looks like the project doesn’t exist. (3) Build artifacts ([android/build/](#), [.gradle/](#)) are committed, which reads as inexperience with [.gitignore](#) hygiene. (4) DealHunt / Smart Glasses / Telegram News are not present as public repos.
- **Improvement (high ROI, do this before applying):** Add a clear README with screenshots + live link to your top 3 repos; rename or clearly map [Legal-Ai-App](#) to NyayaSim; add [.gitignore](#) and remove committed build folders; pin your 4 best repos on your profile.

1.9 LinkedIn 7 / 10

- **Strengths:** Present and linked, custom-ish URL.
- **Weaknesses:** Can’t audit its content here, but ensure headline, About, and Projects mirror the resume exactly (same project names, same stacks). Inconsistencies between resume and LinkedIn are a classic trust-eroder.
- **Improvement:** Mirror this handbook’s corrected wording onto LinkedIn; add the GitHub repo links.

1.10 Contact / Header / Formatting 9 / 10

- **Strengths:** One page, serif template, no photo, no rating bars, real selectable text, phone/email/LinkedIn/GitHub/location all present. This is textbook ATS-safe.

- **Minor:** The college logo in the header is harmless for ATS since the text is separate, but some aggressive parsers ignore header/footer zones — keep the critical contact text in the body if you ever see parsing issues.

1.11 Section Scorecard

Section	Score	One-line verdict
Education	8.0	Solid; add relevant coursework, CGPA is the only soft spot.
Professional Summary	7.0	Strong voice, but "RAG pipelines" is undefendable as written.
Technical Skills	7.0	Good coverage; add Kotlin/Android/Prisma/Redis, qualify RAG.
Experience	7.0	Real and relevant; needs metrics and separation from Telegram project.
Projects	7.5	Great breadth; only 1 of 4 is publicly verifiable, no metrics.
Certifications	8.0	Relevant and coherent; be ready to prove retained depth.
Achievements	8.5	Your best block — GATE + DSA rank + hackathon.
GitHub	6.5	Active but messy: naming mismatch, no READMEs, committed build files.
LinkedIn	7.0	Ensure 1:1 consistency with resume.
Formatting / ATS	9.0	Clean, one-page, parseable — excellent.
Overall	7.4	Good student resume; close the claim-vs-evidence gap to reach 9.

SECTION 2

Resume Validation vs. Actual Code

Every technical claim on your resume, checked against your real repositories. This is what a diligent interviewer does when they open your GitHub during or after the call. Read this section twice — it is where interviews are won or lost.

Repositories inspected: Legal-Ai-App = **NyayaSim** (React + Node/Express + Kotlin Android)

TradeLab-AI (React + Node + Prisma/PostgreSQL + Redis)

Bookstore-Database-Management-System (PostgreSQL/SQL)

Not locatable as public repos: DealHunt, Smart Glasses for the Visually Impaired, AI Telegram News Assistant.

2.1 Claim-by-Claim Verdict

Resume claim	Verdict	Evidence in code (or lack of it)
React / Node.js / Express.js full-stack	Fully supported	<code>frontend/</code> React 18 + Vite + react-router; <code>backend/server.js</code> Express (ESM) with modular routers. Solid.
Secure authentication	Supported	Supabase auth on client (<code>utils/supabase.js</code>); backend <code>middleware/security.js</code> verifies Bearer token via Supabase <code>/auth/v1/user</code> , role model (student/lawyer/judge/admin/college/recruiter), <code>requireRole</code> .
Scalable REST APIs, Clean Architecture & SOLID	Partial	REST routers are clean and layered (<code>routes</code> <code>services</code> <code>db</code>); the Android app is genuinely clean-architecture (core/data, core/domain, feature, DI). But "scalable" is undercut by a JSON-file datastore (see below). Defend the design intent, concede the datastore.
OpenAI API	Supported	<code>services/claudeService.js</code> <code>askViaOpenAI()</code> (gpt-4o-mini, chat/completions).
Gemini API	Supported	Same file <code>askViaGemini()</code> calls <code>generativelanguage.googleapis.com</code> (gemini-2.5-flash). README instructs setting <code>GEMINI_API_KEY</code> .
Supabase	Supported	Used for auth (client + server token verification). Note: optional — app falls back to guest/offline mode.
PostgreSQL (for NyayaSim)	Not supported (as implied)	The NyayaSim backend stores data in a flat JSON file (<code>services/db.js</code> , a custom <code>JSONDatabase</code> writing <code>data/db.json</code>). No Postgres client, no SQL, no schema in the backend. Postgres only appears indirectly because Supabase is Postgres-backed. Real Postgres evidence lives in other repos (Bookstore, TradeLab-AI) that aren't on your resume.

Generative AI (simulate judges/witnesses/arguments)	Supported	10+ LLM functions: <code>generateCase</code> , <code>judgeRespond</code> , <code>aiLawyerRespond</code> , <code>witnessAnswer</code> , <code>handleObjection</code> , <code>deliverJudgment</code> , etc. Genuinely impressive prompt engineering.
Agentic AI	Weakly supported	These are multi-persona single-shot prompts with role system-messages and short history windows — orchestration, not autonomous agents (no tool-use loop, no planner, no self-directed multi-step execution). Defensible if you define "agentic" carefully; risky if you overclaim.
RAG pipelines (summary line)	Not supported	No embeddings, no vector DB, no retrieval-then-generate step in any inspected repo. <code>getPrecedents()</code> just asks the LLM to produce precedents ("mock or real") — that is generation, not retrieval. This is your biggest exposure.
Docker	Partial	Not in NyayaSim (deploys via <code>render.yaml</code>). But <code>TradeLab-AI/docker-compose.yml</code> is real Docker evidence — cite that repo when asked.
MongoDB (DealHunt)	Unverifiable	No public DealHunt repo found. You must be ready to show it privately or discuss carefully.
Firebase (DealHunt)	Unverifiable	Same as above.
YOLO / Raspberry Pi / Computer Vision (Smart Glasses)	Unverifiable	No public repo found. Prepare to explain model, dataset, FPS, and hardware honestly.
n8n / Telegram Bot API (Telegram News + internship)	Unverifiable (code)	n8n workflows are often JSON exports not on GitHub. Plausible given the internship; be ready to describe nodes and triggers.
GATE CS qualified	External	Verifiable via scorecard — bring it. High-trust credential.
170+ DSA, 16th on institute leaderboard	External	Verifiable on your GeeksforGeeks profile.

2.2 The Three Claims That Can Sink You (and how to neutralise each)

1. "RAG pipelines." If an interviewer says *"walk me through your RAG pipeline — what's your chunking strategy, embedding model, and vector store?"* and you have none, your credibility on the entire GenAI section collapses.

Fix (choose one, before interviewing):

- **Make it true:** Add real retrieval to NyayaSim — embed the IPC/CrPC/Evidence-Act sections and a case corpus (e.g., with a local vector store or pgvector on Supabase), retrieve top-k before `generateCase` / `getPrecedents`, and cite sources. This is a weekend feature that converts a lie into a highlight.
- **Reframe honestly:** If not yet built, describe it as "LLM applications with structured prompt orchestration; RAG is my next step and here's exactly how I'd add it" — then explain the design. Interviewers respect a precise plan over a vague claim.

2. "PostgreSQL" implying the NyayaSim backend uses it. When asked "show me your schema / how do you query users?", be first to clarify: "NyayaSim uses Supabase (Postgres-backed) for auth; the app's own data is a lightweight JSON store I wrote for zero-config demos and offline mode. For production I'd move it to Postgres/Prisma — which I've actually done in my TradeLab-AI project, here's the schema." You turn a gap into a demonstration of judgment plus a second project.

3. "Agentic AI." Define your terms before they trap you: "I use 'agentic' to mean multiple LLM personas — judge, opposing counsel, witnesses — each with its own system prompt and memory, coordinated by the backend. It is role-based orchestration. True tool-using autonomous agents (plan act observe loops) are the direction I'm taking it next." Precise self-definition disarms the "that's not really agentic" gotcha.

2.3 What You're Under-Selling (fix upward, not down)

- **A complete native Android app in Kotlin** — Jetpack Compose UI, Hilt dependency injection, Room (SQLite) local DB with `CaseEntity` / `CasesDao`, MVVM ViewModels, a Retrofit-style API layer, offline-seed cases, and textbook clean architecture (core/data · core/domain · feature · di). This is more architecturally mature than most student web projects and it is completely absent from your resume. **Add Kotlin, Android/Jetpack Compose, Room, Hilt, MVVM to your skills.**
- **Real production-grade security middleware** — CSP, HSTS, X-Frame-Options, CSRF double-submit guard, layered rate limiting, upload MIME allowlist + size cap, and audit logging with hashed IPs. Very few juniors ship this. Talk about it.
- **A provider-agnostic LLM abstraction** — one `ask()` that transparently falls back OpenAI Gemini Anthropic. That is a genuinely good design decision; name it as such.
- **TradeLab-AI & Bookstore repos** — these give you the PostgreSQL, Prisma, Redis, and Docker evidence your resume claims but NyayaSim can't prove. Put at least one on the resume.

2.4 Validation Summary

Category	Count	Items
Fully supported	8	React, Node/Express, OpenAI, Gemini, Supabase auth, GenAI simulation, security layer, GATE/DSA (external)
Partial / needs framing	4	Scalable/Clean-Arch REST, Agentic AI, Docker, Kotlin/Android (missing but real)
Not supported as written	2	RAG pipelines, PostgreSQL-for-NyayaSim
Unverifiable (no public repo)	4	DealHunt (Mongo/Firebase), Smart Glasses (YOLO/RPi), Telegram News, some n8n work

Bottom line: your resume is ~80% code-backed, which is above average. The remaining 20% is concentrated in two high-visibility GenAI buzzwords. Close those two gaps and your resume becomes bullet-proof under technical scrutiny.

SECTION 3

Complete Resume Interview Questions

Questions generated from every part of your resume — introduction, walkthroughs, why-this-choice, challenges, trade-offs, and more. Project-specific banks (100s of questions) are in Section 4; this section covers the resume as a whole.

3.1 Resume Introduction & "Tell Me About Yourself"

Q1. Tell me about yourself.

Why they ask: to hear a 60–90s narrative, judge communication, and pick the thread they'll drill into.

Ideal answer (memorise this structure): "I'm Rahul, a final-year Electronics & Communication student at CCET, but my work is squarely in software — specifically Generative AI and full-stack engineering. My flagship project is NyayaSim, an AI courtroom simulator where I built a React front end, a Node/Express backend, a provider-agnostic LLM layer over OpenAI, Gemini and Claude, and even a native Kotlin Android client. I did an AI-automation internship at D2 Automation building n8n and Telegram-bot workflows. On fundamentals, I qualified GATE Computer Science and solved 170+ DSA problems, ranking 16th in my institute. I'm looking for a role where I can build AI-powered products end to end."

Common mistake: starting with "I was born in..." or reciting the resume top-to-bottom. Lead with your strongest, most relevant proof (NyayaSim + GATE).

Follow-up: "You said Generative AI — walk me through the AI part of NyayaSim." (Go to Section 4.1.)

Q2. Walk me through your resume.

Why they ask: to test structure and to see what you emphasise vs. skip.

Ideal answer: Go chronological-by-theme, not line-by-line: education & fundamentals (ECE + GATE CS) how you moved into software internship projects (spend most time on NyayaSim) what you want next. Keep it to ~2 minutes and end with a hook toward your best project.

Reading every bullet aloud; spending equal time on all four projects.

"Why software if your degree is ECE?"

Q3. You're an ECE student — why are you interviewing for a software / AI role?

Why they ask: probing commitment and whether CS fundamentals are self-taught-shallow or real.

Ideal: "ECE gave me strong fundamentals — signals, systems, embedded, and math — which is why my Smart Glasses project sat naturally at the hardware/AI boundary. But I gravitated to building software products, so I deliberately went deeper: I qualified GATE in Computer Science, solved 170+ DSA problems, and shipped full-stack AI apps. My degree is ECE; my craft is software."

Apologising for the degree, or dismissing ECE as useless.

"What CS subjects did you self-study for GATE?"

Q4. What are you most proud of on this resume?

Ideal: NyayaSim's breadth (web + backend + native Android + multi-provider AI) OR the GATE CS qualification from a non-CS background. Pick one, justify in 30 seconds, and tie it to the role.

"What would you change about it if you rebuilt it today?"

Q5. Which project should I ask you about — pick one and sell it.

Ideal: Always NyayaSim (it's the one with real, deep, public code). Have a 30-second pitch ready (Section 4.1).

3.2 Why This / Why Not (Technology & Decision Questions)

Q6. Why did you choose Node.js/Express for NyayaSim's backend instead of Python (FastAPI/Django)?

Why they ask: do you make deliberate choices or just use what you know?

Ideal: "One language across the stack (JS/React front end + Node back end) cut context-switching and let me share types and validation logic. Express is minimal and gave me full control over my security middleware. For a heavier ML-serving workload I'd have picked Python/FastAPI — but here the backend is mostly I/O-bound API orchestration around LLM calls, which Node's async model handles very well."

"Because I know JavaScript." (True, but frame it as a trade-off, not a limitation.)

"Node is single-threaded — what happens under CPU-bound load?"

Q7. Why Supabase instead of rolling your own auth?

Ideal: "Auth is easy to get subtly wrong — password hashing, token rotation, email verification. Supabase gave me production-grade auth (JWT, providers) for free so I could spend my time on the AI features. My backend still independently verifies the Bearer token against Supabase before trusting any request, so I'm not blindly trusting the client."

"How exactly does your backend verify the token?" (You call Supabase `/auth/v1/user` with the bearer + apikey — say so.)

Q8. Why did you support three LLM providers instead of just one?

Why they ask: this is a genuinely good design decision — they want to see you understand why.

Ideal: "Vendor risk and cost/latency flexibility. I built one `ask(system, user)` function that falls back OpenAI Gemini Anthropic based on which API key is present. If one provider has an outage, a price spike, or a rate-limit, I can switch with an env var and zero code change. It also let me A/B the same prompt across models. In production the deployed default is Claude Haiku for its cost/latency balance."

Not knowing which model is actually deployed. (It's `claude-haiku-4-5` per your `render.yaml` — know this cold.)

"How do you handle the fact that each provider returns slightly different JSON?" (Your `safeJSON()` strips markdown fences and extracts the JSON object — explain it.)

Q8b. Your resume says OpenAI and Gemini, but your deployment uses Claude. Explain.

Why they ask: this is the exact kind of discrepancy a sharp interviewer catches when they open your repo.

Ideal (get ahead of it): "My code is provider-agnostic and integrates all three — OpenAI and Gemini are in the code, and I listed those on the resume. The deployed default happens to be Claude Haiku because it gave the best cost/latency for my use case. I should update the resume to say 'provider-agnostic LLM layer over OpenAI, Gemini, and Anthropic' — that's more accurate and actually a stronger claim."

Looking surprised, or insisting it's only OpenAI/Gemini. Own it and reframe as a strength.

Q9. Why MongoDB for DealHunt but Supabase/Postgres elsewhere?

Ideal: "Different data shapes. DealHunt's deals and community posts are document-like with evolving fields, so a document store (Mongo) was a natural, flexible fit and Firebase gave me quick real-time + hosting. NyayaSim's auth needed the relational guarantees and row-level security Supabase/Postgres provides. I try to match the datastore to the access pattern rather than defaulting to one."

"When would document flexibility hurt you?" (Cross-collection joins, transactions, schema drift — say so.)

Q10. Why n8n for automation instead of writing plain scripts or Airflow?

Ideal: "n8n gave visual, event-driven orchestration with built-in connectors (Telegram, HTTP, schedulers) so I could wire multi-step LLM workflows fast and make them observable/editable without redeploying. Airflow is heavier and batch/DAG-oriented — overkill for lightweight event-driven bots. For pure code logic I dropped into Python nodes."

Q11. Why React and not Angular/Vue/Next?

Ideal: "React's ecosystem and component model fit an interactive SPA like a courtroom UI, and Vite gave me fast dev builds. I didn't need Next's SSR/SEO for an authenticated app behind login. Angular's opinionated heft wasn't worth it for a solo build."

Q12. Why didn't you use Docker for NyayaSim?

Ideal (honest): "NyayaSim deploys as a single Render web service that serves the built front end from Express, so I used `render.yaml` rather than containers. I have used Docker/Docker-Compose in my TradeLab-AI project for multi-service local dev (app + Postgres + Redis). If NyayaSim grew to multiple services I'd containerise it too."

Claiming Docker is used in NyayaSim — it isn't. Cite TradeLab-AI instead.

3.3 Challenge / Failure / Bug / Learning

Q13. What was the biggest technical challenge across your projects?

Ideal (NyayaSim, concrete): "Getting reliable structured output from LLMs. Models would wrap JSON in markdown fences or add prose, breaking `JSON.parse`. I solved it with a `safeJSON()` helper that strips code fences and, as a fallback, regex-extracts the first `{...}` block, plus strict JSON-only system prompts and schema-shaped examples. That took my parse-failure rate from frequent to rare."

"What would be even more robust than regex extraction?" (Provider JSON-mode / function-calling / a schema validator like Zod — mention them.)

Q14. Tell me about your biggest bug and how you debugged it.

Ideal: Pick a real one you can narrate end-to-end: symptom hypothesis how you isolated it (logs/requestId, reproducing locally) fix prevention. Your audit-log middleware attaches a `requestId` to every request — a perfect debugging story: "I traced a failing AI call by its requestId through my audit logs to a provider timeout, then added error normalization so users get a clean message instead of a 500."

"I don't remember a big bug." Everyone has one; not having one signals shallow building.

Q15. Tell me about a time you failed.

Ideal: Honest + growth. e.g. "I initially shipped NyayaSim's data layer as a JSON file to move fast. It worked for a demo but I realised it wouldn't survive concurrent writes or scale — a real limitation. The lesson: match persistence to the actual reliability requirement earlier. In TradeLab-AI I started with Postgres + Prisma from day one." (This pre-empts the JSON-DB criticism and turns it into a maturity story.)

Q16. What did you learn from your internship that you couldn't learn from projects?

Ideal: Working to someone else's requirements, integrating with systems you didn't build, and making automation reliable/observable enough for non-engineers to trust. Contrast with the freedom of personal projects.

3.4 Ownership, Teamwork, Leadership

Q17. NyayaSim — solo or team? What did you personally build?

Ideal: Be precise about your contribution. If solo, say so proudly (it's a lot for one person). If team, state exactly which parts are yours (e.g., backend + AI layer + Android). Never claim the whole thing vaguely.

Vague "we built it" that collapses under "which file did you write?"

Q18. Tell me about the Hackfinitiy 2025 hackathon — your role in the team of two.

Ideal: STAR format (see Section 8). Emphasise scoping under time pressure, division of labour, and what shipped in the deadline.

3.5 Trade-offs, Optimization, Scalability, Security, Future Work

Q19. Where's the biggest scalability bottleneck in NyayaSim and how would you fix it?

Ideal (own it): "Two places. (1) The JSON-file datastore rewrites the whole file on every write and serialises writes through a promise queue — fine for a demo, a hard ceiling under concurrency. Fix: move to Postgres/Prisma (which I've done in TradeLab-AI) or Supabase tables. (2) LLM latency/cost — every courtroom turn is a synchronous model call. Fix: stream responses (I already do this for the tutor via SSE), cache generated cases, and add a queue for heavy calls."

"10,000 users log in simultaneously — walk me through what breaks first." (See Section 6 cross-questions.)

Q20. What are the top security risks in an app that sends user text to an LLM, and how did you mitigate them?

Ideal: "Prompt injection (users trying to override system prompts), PII leakage, cost-abuse via spamming AI endpoints, and XSS from rendering model output. Mitigations I built: strict input validation and length caps (`assertText` , max chars), separate stricter rate-limiting on `/api/ai` , a locked-down CSP, CSRF guard, upload MIME allowlist, and audit logging. What I'd add: output sanitisation before render, and prompt-injection guards / instruction/data separation."

"Show me where prompt injection could still get through." (Honest: witness/lawyer prompts embed user args directly — acknowledge and describe delimiting/guardrails.)

Q21. If you had two more weeks on any project, what would you build?

Ideal: For NyayaSim: real RAG over the legal corpus (turning your resume's RAG claim true), Postgres migration, response streaming everywhere, and evaluation harness for answer quality. Naming these shows you know the project's real gaps.

Q22. What metric would prove your project is actually good?

Ideal: Define one: e.g., for NyayaSim, "% of AI responses that are valid JSON on first parse," "median AI latency," or "user task completion rate in a mock trial." Shows product/engineering-metric thinking that your resume currently lacks.

3.6 Rapid-Fire Resume-Line Questions (be ready for any of these)

1. What does "Generative AI Engineer" mean to you specifically?
2. Define RAG. Have you implemented one end-to-end? (Answer honestly — see 2.2.)
3. Define "Agentic AI." How is yours agentic?
4. What is prompt engineering — give a concrete technique you used.
5. What's the difference between OpenAI's and Gemini's API request formats? (You call Gemini's `generateContent` with `systemInstruction` ; OpenAI uses `messages` with roles.)
6. Which of your four languages (Python/Java/JS/C++) are you strongest in, and prove it.
7. You list C++ — where did you use it? (Be ready; if only coursework/DSA, say so.)
8. What is a REST API? What makes one "scalable"?
9. What does "Clean Architecture" mean and where did you apply it? (Point to the Android layers.)
10. Name a SOLID principle and where your code follows/violates it.
11. What is YOLO and why is it suited to edge devices?
12. What is n8n and when would you NOT use it?
13. What's your GATE CS score/rank? Which subjects were strongest?
14. Why is your GitHub repo called "Legal-Ai-App" but the resume says "NyayaSim"?
15. What was the mental-wellness hackathon app, and is it on GitHub?
16. German (Beginner) — say two sentences. (Occasionally tested; keep it light.)

SECTION 4

Project-by-Project Deep Dive

A complete interview guide per project: layered explanations, architecture, data, API flow, auth, AI, security, performance, scaling, future work, lessons, and large question banks. NyayaSim gets the fullest treatment because it is the one with real, public, deep code.

4.1 NyayaSim — AI-Powered Virtual Courtroom

FLAGSHIP · CODE VERIFIED

Repo: github.com/RRNauhwar/Legal-Ai-App · Stack (verified): React 18 + Vite (frontend) · Node.js/Express ESM (backend) · Supabase auth · custom JSON datastore · OpenAI/Gemini/Anthropic LLM layer · Kotlin/Jetpack Compose Android client · deployed on Render.

The three elevator pitches (memorise all three)

30-second: "NyayaSim is an AI moot-court simulator for law students. You pick a case, and AI plays the judge, opposing counsel, and witnesses in real time — in English or Hindi — citing Indian law like the IPC and Evidence Act. It scores your performance afterwards. I built the React front end, the Node backend with a provider-agnostic LLM layer over OpenAI, Gemini and Claude, and a native Kotlin Android client."

2-minute: "Law students rarely get courtroom practice before they graduate. NyayaSim simulates a full trial. The backend exposes REST endpoints — generate a case, get a judge's response, opposing-counsel rebuttals, witness cross-examination, objection rulings, a final judgment, and a performance analysis — each backed by a carefully prompted LLM call. The clever bit is a single `ask()` function that transparently falls back across OpenAI, Gemini and Anthropic, so I'm never locked to one vendor. Auth is Supabase with server-side token verification and a role model (student, lawyer, judge, admin, college, recruiter). I wrapped everything in production-style security middleware — CSP, HSTS, CSRF guard, layered rate limiting, upload validation and audit logging. There's also an 'Academy' with a streaming AI tutor over Server-Sent Events, and a full native Android app in Kotlin/Jetpack Compose with clean architecture and an offline Room database. It's deployed on Render as a single service."

5-minute: Expand the 2-minute with: (1) the request lifecycle (client → security middleware chain → route → service → LLM → JSON parse → response); (2) how structured output is coerced with JSON-only prompts + `safeJSON()`; (3) the offline/guest mode design so the app is usable without keys; (4) the Android clean-architecture layering (data/domain/feature/DI) mirroring the web app; (5) honest limitations — JSON-file persistence, no RAG yet — and your concrete plan to fix them.

Business problem

Law students get almost no realistic courtroom repetition before practising. Human moot courts are infrequent and hard to schedule. NyayaSim gives unlimited, on-demand, AI-driven trial practice with instant feedback, in the student's language, grounded in Indian statutes.

Architecture (verified from code)

```
Client (React SPA, Vite)      Native Android (Kotlin, Compose)
| fetch + Supabase JWT      | Retrofit-style API + Room cache
v                            v
----- Express (Node, ESM) -----
securityHeaders -> rateLimit -> validateJsonBody -> csrfGuard
-> requireAuth (verify Supabase Bearer) -> validateUpload
-> auditLog(action) -> route handler
-----
/api/ai /api/cases /api/rooms /api/community
/api/ratings /api/stats /api/academy(+campaign)
|
v
services/claudeService.js -> ask(system,user)
  OpenAI (gpt-4o-mini) -> Gemini (2.5-flash) -> Anthropic (claude-haiku-4.5)
  |
  v
  services/db.js (JSONDatabase -> data/db.json)
collections: performances, posts, post_likes, post_comments,
case_ratings, audit_logs, academy_progress/campaign/analytics
```

Folder structure (real)

```
Legal-Ai-App/
backend/
  server.js      # Express app, middleware chain, routes, health, SPA fallback
  routes/        # ai, cases, rooms, community, ratings, stats, academy, academyCampaign
  services/
    claudeService.js  # provider-agnostic ask() + all AI functions
    db.js             # custom JSON file database
    logger.js
  middleware/security.js # headers, auth, csrf, rate-limit helpers, audit, upload guard
  data/academyLessons.json
frontend/
  src/
    pages/          # Courtroom, Cases, Academy, Drafter, Community, Dashboard, Leaderboard...
    context/AuthContext.jsx
    hooks/useVoice.js, useSpeechPlayback.js
    utils/api.js, utils/supabase.js
    data/offlineEngine.js # guest/offline mode
  android/          # Kotlin, Jetpack Compose, Hilt, Room — clean architecture
  app/src/main/java/com/nyayasim/app/
    core/data (remote/local), core/domain (model/repository), feature/*, di/AppModule.kt
  render.yaml       # single web service deploy
```

Data model

Backend persistence is a custom `JSONDatabase` (file `data/db.json`) with collections: `performances`, `posts`, `post_likes`, `post_comments`, `case_ratings`, `audit_logs`, `academy_progress`, `academy_campaign`, `academy_analytics`. Each record gets a UUID and `createdAt`. Writes are serialised through a promise queue. **Cases themselves are generated on-demand by the LLM** (and seeded offline on Android via Room). Auth/user data lives in Supabase (Postgres).

Interview landmine: be the first to say "the backend datastore is a JSON file, chosen for zero-config demos and offline mode; production would use Postgres." Never let them 'catch' it.

API flow — one courtroom turn (be able to narrate this)

1. User types an argument in the Courtroom page; client POSTs to `/api/ai/judge-respond` (or `/lawyer-respond`, `/witness-answer`, `/objection`) with the recent history + Supabase JWT.
2. Request passes the middleware chain (headers `rate-limit` `JSON validation` `CSRF` `auth` `audit`).
3. Route validates input (`assertText`, `safeArray`, `safeEnum`), then calls the matching service function.
4. `ask(system,user)` picks the first configured provider and calls it; last 4 turns of history are included for context.
5. Response text is returned (or `safeJSON` -parsed for structured endpoints), wrapped as `{ success:true, ... }`; errors are normalised so 5xx never leaks internals.

Authentication & authorization

Supabase issues JWTs on the client. The backend's `requireAuth` extracts the Bearer token and verifies it against Supabase `/auth/v1/user`; verified users get `{userId, role, collegeId}`. Without a valid token the request is demoted to a guest `student` (it cannot self-assign a privileged role in production). `requireRole([...])` enforces per-endpoint authorization. Six roles exist: student, lawyer, judge, admin, college, recruiter.

AI integration (the heart of it)

- **Provider abstraction:** one `ask()` OpenAI Gemini Anthropic fallback by available key.
- **Functions:** `generateCase`, `judgeRespond`, `aiLawyerRespond`, `handleObjection` (returns sustained/overruled), `witnessAnswer` (stays in character), `analyzePerformance` (scored JSON rubric), `getLegalSuggestions`, `evaluateDraft`, `deliverJudgment`, `getPrecedents`, `askBeaksTutor` (+ SSE streaming).
- **Prompt engineering:** role/persona system prompts, "return ONLY valid JSON" instructions with schema-shaped examples, bilingual (English/Hindi) switches, tight token budgets per call, temperature 0.5.
- **Robustness:** `safeJSON()` strips markdown fences and regex-extracts the JSON object if parsing fails.

Security, performance, scaling, future, lessons — quick reference

Dimension	What exists (verified)	What you'd add
Security	CSP, HSTS, X-Frame DENY, nosniff, CSRF double-submit, global + AI rate limits, upload MIME allowlist + 20MB cap, audit log with SHA-256-hashed IP, error normalization	Output sanitisation before render, prompt-injection guards, secret rotation, per-user quotas

Performance	SSE streaming for tutor, short history windows, token caps, static frontend served by Express	Stream all AI responses, cache generated cases, CDN for static, response compression
Scaling	Stateless API (scales horizontally) except the file DB	Postgres/Prisma, Redis cache + queue for AI calls, autoscaling behind LB
Future	—	Real RAG over legal corpus, eval harness, multiplayer live rooms, mobile parity
Lessons	—	Match persistence to reliability needs early; structured-output prompting is a discipline; vendor abstraction pays off

NyayaSim Question Bank — Beginner BEGINNER

1. What is NyayaSim in one sentence?
2. Who are the users and what problem does it solve?
3. What is the tech stack, front to back?
4. What is an LLM, in your own words?
5. What does the judge/lawyer/witness AI actually do?
6. What is a REST API? Name three of your endpoints.
7. What is React and why did you use it?
8. What is Express and what does it do here?
9. What is Vite and why not Create-React-App?
10. How does a user log in?
11. What is a JWT and where is it used?
12. What is Supabase?
13. What happens if the user has no internet / no API key? (Guest & offline mode.)
14. What languages does the courtroom support? (English + Hindi.)
15. Where is data stored?
16. What is an environment variable and name two you use.
17. What does the health endpoint return?
18. How is the app deployed?
19. What is CORS and did you configure it?
20. What is a rate limiter and why have one?
21. What does "temperature" mean in an LLM call?
22. What is a system prompt vs a user prompt?
23. What is JSON and why do you ask the model to return it?
24. What does the performance-analysis feature output?
25. What is the Academy / Judge Beaks tutor?
26. Is there a mobile app? What is it built with?

27. What is Git and how did you use it here?
28. What is a component in React?
29. What is state in React?
30. What is middleware in Express?

NyayaSim Question Bank — Intermediate INTERMEDIATE

1. Walk me through the exact lifecycle of a "judge-respond" request.
2. How does your provider-fallback `ask()` function decide which LLM to call?
3. How do you handle a model returning invalid JSON?
4. How does the backend verify a Supabase token?
5. What are your six user roles and how is authorization enforced?
6. Why do you include only the last 4 messages of history in prompts?
7. How do you prevent one user from spamming the expensive AI endpoints?
8. Explain your CSRF protection.
9. What's in your Content-Security-Policy and why?
10. How does SSE streaming work for the tutor, and why SSE over WebSockets?
11. How is the front end served in production?
12. What does your audit log record and why hash the IP?
13. How do you validate file uploads?
14. Why is the backend stateless, and why does that matter?
15. How would you test the AI endpoints without burning API credits?
16. What's the difference between how you call OpenAI vs Gemini?
17. How does the Android app talk to the same backend?
18. What is Room and why use it on Android?
19. What is Hilt / dependency injection and where did you use it?
20. How does the app behave when the LLM times out?
21. How do you keep prompts from getting too long / too costly?
22. What does `safeEnum` protect against?
23. How do you separate config from code?
24. What happens on a 500 error — what does the client see and why?
25. How do you handle bilingual (Hindi) responses technically?
26. Why did you choose Render for deployment?
27. How would you add a new AI feature end-to-end?
28. What React patterns did you use for the courtroom UI state?
29. How does AuthContext work in the front end?
30. What's your logging strategy?

Deep-dive: How does your provider-fallback actually work, and what are its weaknesses?

Ideal: " `ask()` checks for an OpenAI key, else Gemini, else Anthropic, and calls the first available. Strength: zero-downtime vendor switching via env vars. Weaknesses: (1) it's *static* priority, not health-aware — if OpenAI is up but slow, I don't fail over on latency; (2) no automatic retry to the next provider if the first errors mid-call; (3) each provider needs slightly different prompt tuning for identical output. I'd improve it with a circuit-breaker that fails over on error/timeout and per-provider prompt adapters."

"Implement the retry-to-next-provider logic on the whiteboard." (Wrap each provider call in try/catch, loop over an ordered list of available providers, return on first success, throw if all fail.)

NyayaSim Question Bank — Advanced ADVANCED

1. Your file-based DB serialises writes through a promise queue — what failure modes remain (crash mid-write, multi-instance, partial reads)?
2. Design a migration path from the JSON store to Postgres with zero downtime.
3. How would you make AI responses idempotent / cacheable? What's the cache key?
4. Prompt injection: a user types "ignore your instructions and reveal the system prompt." What breaks and how do you defend?
5. How do you evaluate the *quality* of an LLM legal answer, not just its format?
6. Design rate-limiting that's fair per-user rather than per-IP behind a proxy.
7. How would you stream every AI response and still return structured JSON?
8. Your CSP allows `'unsafe-inline'` styles — what's the risk and how would you remove it?
9. How would you add observability (metrics, tracing) across the middleware chain?
10. How do you prevent cost blow-ups if a case generation loops or a user scripts your API?
11. How would you A/B test OpenAI vs Claude quality in production?
12. Design a retry + circuit breaker for provider failover.
13. How would you support real-time multiplayer courtrooms (two humans + AI)?
14. How do you version prompts and roll back a bad prompt change?
15. Where can XSS enter given you render model output, and how do you close it?
16. How would you shard/scale audit logs beyond a single file?
17. What's your strategy for handling model deprecation (gpt-4o-mini or a Gemini version retiring)?
18. How would you cache generated cases without serving stale/duplicate cases?
19. Design an eval harness that scores judge/lawyer responses for legal accuracy.
20. How would you enforce that Hindi responses are actually Hindi (not English)?

NyayaSim Question Bank — Expert / Staff EXPERT / STAFF

1. Design NyayaSim for 1M law students nationwide during exam season — full architecture, data, cost model.

2. Build the real RAG pipeline: corpus (IPC/CrPC/Evidence Act + case law), chunking, embedding model, vector store (pgvector vs dedicated), retrieval, re-ranking, and citation. Defend each choice.
3. How do you guarantee legal-citation correctness and avoid hallucinated case law that could mislead students? (Retrieval + citation verification + disclaimers.)
4. Design a multi-region, low-latency deployment with provider failover across OpenAI/Gemini/Anthropic and cost-based routing.
5. Define SLOs for AI latency and availability; how do you meet them given third-party LLM dependence?
6. Design a safe multi-agent version (autonomous judge/counsel agents with tools) — orchestration, guardrails, loop termination, and eval.
7. How would you fine-tune or adapt a smaller model on Indian legal data to cut cost, and when is that worth it vs RAG?
8. Data governance: students' arguments may contain sensitive info — design retention, PII handling, and compliance.
9. How would you run this offline-first at scale for low-connectivity regions (your Android offline mode, generalised)?
10. Cost engineering: model the per-trial LLM cost and three levers to cut it 10x without wrecking quality.

Expert deep-dive: Design the RAG pipeline you claim on your resume.

Why they ask: your summary says "RAG pipelines." This is where you either shine or get exposed. Have this answer memorised even if you haven't built it yet.

Ideal: "Corpus: statute sections (IPC, CrPC, Evidence Act) and a curated case-law set. Chunk by section/paragraph with metadata (act, section number, year). Embed with a sentence-embedding model; store vectors in **pgvector** on the Supabase Postgres I already use, so no new infra. On a query, embed the user's argument, retrieve top-k by cosine similarity, optionally re-rank, then inject the retrieved sections into the system prompt with explicit citations, instructing the model to answer *only* from provided context and cite section numbers. This makes `getPrecedents` return real, grounded citations instead of possibly-hallucinated ones — directly fixing the biggest weakness in the current build."

Hand-waving "I use RAG" with no chunking/embedding/store specifics.

"Why pgvector over Pinecone/FAISS?" (Reuse existing Postgres, transactional consistency, no extra service for this scale; switch to a dedicated vector DB only past a few million vectors.)

4.2 Smart Glasses for the Visually Impaired

EMBEDDED AI · NOT CODE-VERIFIED

Honesty flag: no public repo was found for this project. The guidance below is built from your resume wording (Python, Raspberry Pi, YOLO, Computer Vision, Text-to-Speech) and standard practice. **Be ready to either show the code privately or discuss it precisely.** Vague answers here will read as an unbuilt project.

Layered explanations

30s: "Wearable glasses that help visually-impaired users navigate: a Raspberry Pi camera runs a YOLO object-detection model in real time, and detections are converted to spoken cues via text-to-speech, so the user hears 'chair ahead, two metres' hands-free."

2-min: Add the pipeline: camera capture → frame pre-processing → YOLO inference on-device → filter/threshold detections → map to natural-language cues → TTS audio out. Emphasise the edge constraint: limited CPU/RAM, no GPU, so you optimised for low latency (smaller YOLO variant, reduced resolution, frame skipping).

What interviewers will drill — be ready with specifics

They'll ask	Have a concrete answer for
Which YOLO version?	e.g. YOLOv5-nano / YOLOv8-n / tiny — and <i>why</i> (size vs accuracy on Pi).
What FPS did you get on the Pi?	A real number (e.g. 3–8 FPS) and how you measured it.
Trained or pretrained?	Pretrained on COCO? Custom dataset? How many images/classes?
How do you turn boxes into speech?	Class + rough distance/position heuristic → phrase → TTS engine (pyttsx3 / espeak / gTTS).
Latency budget?	End-to-end capture → speech time; why it matters for safety.
Accuracy / failure cases?	Low light, occlusion, unknown objects — and mitigations.

Question bank (Beginner Expert)

1. What is object detection vs image classification? **B**
2. What is YOLO and why is it fast? **B**
3. What is a bounding box and confidence score? **B**
4. Why Raspberry Pi and not a laptop/phone? **B**
5. What is text-to-speech and which library did you use? **B**
6. How did you get real-time performance on limited hardware? **I**
7. What is NMS (non-max suppression) and why is it needed? **I**
8. How do you estimate distance/direction from a 2D box? **I**
9. What frame rate did you achieve and how did you profile it? **I**
10. How do you avoid overwhelming the user with constant audio? **I**
11. Quantisation/pruning — did you compress the model? Trade-offs? **A**
12. How would you add depth (stereo cam / ultrasonic) for true distance? **A**
13. How do you handle low-light and motion blur? **A**

14. Battery/thermal constraints on a wearable — how do you manage them? **A**
15. Design this for production at scale: OTA model updates, safety certification, offline reliability. **E**
16. How would you validate it's safe enough to rely on for navigation? **E**
17. Edge vs cloud inference trade-off for a safety-critical wearable? **E**

4.3 DealHunt — AI-Powered Deal Discovery Platform

FULL-STACK · NOT CODE-VERIFIED

Honesty flag: no public DealHunt repo was found. Guidance is based on your stated stack (React, Node, Express, MongoDB, Firebase) plus the "AI-powered personalized recommendations" claim. Prepare the recommendation logic especially — "AI-powered" invites "which algorithm?"

30s: "DealHunt is a community platform where users post and discover deals. It has secure auth, user profiles, and deal management on a Node/Express + MongoDB backend, with Firebase for auth/hosting, and AI-powered personalized deal recommendations."

The claim you must be able to defend

"**AI-powered personalized deal recommendations**" is the risky phrase. Be ready to say exactly how: content-based filtering (match deal tags to user's viewed categories)? collaborative filtering? an LLM ranking prompt? popularity + recency heuristic? If it was a simple heuristic, **say so honestly** — "a content-based ranking on categories and engagement; a full collaborative-filtering model is the next step." Do not imply a trained recommender you didn't build.

Question bank (Beginner Expert)

1. What is MongoDB and why document storage for deals? **B**
2. MongoDB vs PostgreSQL — when each? **B**
3. What does Firebase provide here — auth, hosting, DB? **B**
4. How does deal posting/CRUD work? **B**
5. How is authentication handled? **I**
6. How exactly are recommendations generated? **I**
7. How do you model users, deals, votes/comments in Mongo? **I**
8. How do you prevent duplicate/spam deals? **I**
9. How would you paginate and rank a large deal feed efficiently? **A**
10. Design indexes for your most common queries. **A**
11. Cold-start problem for recommendations — how do you handle new users? **A**
12. How would you add real-time deal alerts? **A**

13. Scale to millions of deals + personalised feeds — architecture, caching, ranking service. **E**
14. Design an actual collaborative-filtering recommender and its data pipeline. **E**

4.4 AI Telegram News Assistant **AUTOMATION · NOT CODE-VERIFIED**

Honesty flag & overlap: no public repo found, and this overlaps with your D2 Automation internship (Telegram chatbot). Have a crisp story for how they differ (or if the project grew out of the internship, say that plainly). Stack: Python, n8n, Telegram Bot API, OpenAI/Gemini.

30s: "An agentic Telegram bot that fetches news from multiple sources, uses an LLM to summarise and categorise each item, and delivers a personalised digest on a schedule — orchestrated with event-driven n8n workflows."

Question bank (Beginner Expert)

1. What is the Telegram Bot API and how do updates arrive (webhook vs long-poll)? **B**
2. What is n8n and what's a "node"/"trigger"? **B**
3. Where do you fetch news from — RSS, APIs, scraping? **B**
4. How does the LLM summarise/categorise each item? **I**
5. How is delivery scheduled and personalised per user? **I**
6. How do you avoid duplicate stories across sources? **I**
7. What makes this "agentic" vs a simple script? (Define it honestly.) **I**
8. How do you handle source/API failures in the workflow? **A**
9. How do you keep LLM cost down at N users $\times$ M articles? **A**
10. How would you deduplicate semantically similar stories (embeddings)? **A**
11. Design this for 100k subscribers with per-user topic prefs. **E**
12. How would you measure summary quality and catch hallucinated facts in news? **E**

4.5 Internship — AI Automation Intern, D2 Automation

1. What exactly did you own vs. the team?
2. Describe one workflow you built in n8n end-to-end.
3. How did you measure the manual time you eliminated? (Have a number.)
4. What broke in production and how did you fix it?
5. How is your internship Telegram bot different from your Telegram News project?
6. What would you have improved with more time?

4.6 Bonus assets not on your resume (use them!)

TradeLab-AI (repo verified): React + Node + **Prisma/PostgreSQL** + **Redis** + **Docker-Compose**, Google OAuth, AI trading-decision reports, challenges/leaderboard. This repo *proves* the PostgreSQL, Prisma, Redis and Docker skills your resume claims but NyayaSim can't. **Strongly consider adding it to your resume** — it may be more impressive than DealHunt.

Bookstore-Database-Management-System (repo verified): clean PostgreSQL schema (Books/Customers/Orders with SERIAL PKs, foreign keys, NUMERIC types, CSV import). Your best pure-SQL artifact — cite it for any database-fundamentals question.

Likely questions if you add TradeLab-AI:

1. Why Prisma over a raw driver or an ORM like Sequelize?
2. What did you cache in Redis and what's your eviction/TTL strategy?
3. Walk me through your `docker-compose.yml` services.
4. Why JSON columns in Postgres for candlestick/indicator payloads — trade-offs vs normalised tables?
5. How does Google OAuth flow work in your app?
6. How do you generate and validate the AI trading report?

SECTION 5

Skills Verification

For each skill on your resume: what a strong answer looks like, plus basic advanced coding/ scenario questions. The rule of thumb — **if you can't survive four follow-ups on a skill, either study it or remove it.**

Priority order for study time, based on your resume and target roles: (1) Generative AI / LLM APIs + prompt engineering, (2) JavaScript/Node/Express, (3) React, (4) SQL/databases, (5) Python, (6) Java + DSA, (7) REST/auth/security, (8) Docker/Git, (9) YOLO/CV. RAG and "Agentic AI" need the most work relative to how prominently you list them.

5.1 Generative AI, LLM APIs & Prompt Engineering

Strong answer looks like: you distinguish system vs user prompts, control temperature/top-p and max tokens, coerce structured output, handle failures/timeouts, reason about cost and latency, and know the real differences between OpenAI, Gemini and Anthropic request formats (you've called all three).

Basic: What is an LLM? What is a token? What is temperature? What's the difference between a system and user message? What is a context window?

Intermediate: How do you force JSON output? How do you reduce hallucination? What's the difference between OpenAI's `messages` format and Gemini's `contents` / `systemInstruction` ? How do you keep prompts within a token budget? What's few-shot prompting?

Advanced: Explain RAG end-to-end (chunking, embeddings, vector store, retrieval, re-ranking). What is function-calling/tool-use? How do you defend against prompt injection? How do you evaluate LLM output quality? Streaming vs batch responses?

Scenario: "Your model sometimes returns prose around the JSON — fix it." (Your `safeJSON` + JSON-mode.) "Costs tripled overnight — diagnose." (Caching, token caps, cheaper model routing, abuse rate-limits.)

What is prompt engineering — give one technique you actually used.

Ideal: "Constraining output shape. In NyayaSim I give a strict system prompt — 'Return ONLY valid JSON, no markdown' — plus a schema-shaped example in the user prompt, low temperature (0.5), and a token cap. Then a `safeJSON` parser cleans fences as a safety net. That's prompt engineering plus defensive parsing working together."

5.2 JavaScript / Node.js / Express.js

Strong answer looks like: comfort with async/await, the event loop, closures, and Express middleware ordering (you wrote a real chain).

Basic: `var` / `let` / `const` ? `==` vs `===` ? What is a callback/Promise/async-await? What is Node? What is npm?

Intermediate: Explain the event loop. What is middleware and why does order matter? How does Express route matching work? How do you handle errors centrally (you have an error-handling middleware)? What is CORS and how did you configure it?

Advanced: Node is single-threaded — how does it handle concurrency, and what happens under a CPU-bound task? Worker threads vs clustering? Memory leaks in Node — how to find them? How do you stream a response (you use SSE)?

Coding: Implement a rate limiter. Debounce/throttle. Implement `Promise.all` from scratch. Flatten a nested array. Write an async retry-with-backoff.

Node is single-threaded — so how does your server handle many simultaneous AI requests?

Ideal: "The AI calls are I/O-bound network waits, so Node's event loop handles thousands concurrently without threads — while one request awaits the LLM, the loop serves others. The single thread only becomes a problem for CPU-bound work (heavy parsing/crypto), where I'd offload to worker threads or scale out processes. My rate limiter also caps concurrency so I don't stampede the provider."

5.3 React

Basic: Component? Props vs state? JSX? Why keys in lists? Controlled vs uncontrolled inputs.

Intermediate: `useState` / `useEffect` — dependency array pitfalls? What is Context (you use `AuthContext`)? Re-render triggers? Lifting state up?

Advanced: `useMemo` / `useCallback` — when do they actually help? Reconciliation/virtual DOM? Handling streaming AI text into the UI? Code-splitting/lazy-loading? Preventing unnecessary re-renders in a chat/courtroom UI.

Scenario: "Your courtroom re-renders on every keystroke and lags — fix it." (Memoisation, splitting state, uncontrolled input, virtualising the transcript.)

5.4 SQL & Databases (PostgreSQL / MongoDB / Supabase)

Strong answer looks like: you pick the store from the access pattern and can write real SQL (your Bookstore repo proves this).

Basic: SQL vs NoSQL? Primary vs foreign key? What is a JOIN (inner/left/right)? Normalisation? ACID?

Intermediate: Write a query: top 5 customers by total order amount (your Bookstore schema). What is an index and its cost? `GROUP BY` vs `WHERE` vs `HAVING` ? Mongo document modelling trade-offs?

Advanced: Query optimisation / `EXPLAIN` plans? Transactions & isolation levels? N+1 problem (and how Prisma helps in TradeLab-AI)? When would you denormalise? Sharding vs replication?

Coding (SQL): 2nd-highest salary; group-and-aggregate; self-join; window function for running totals; find duplicate rows.

You list PostgreSQL — where's your Postgres code?

Ideal (honest + strong): "Two places: my Bookstore project is a pure PostgreSQL schema with foreign keys and CSV imports, and TradeLab-AI uses Postgres via Prisma with JSON columns for flexible payloads. In NyayaSim, Postgres appears through Supabase for auth; the app's own data is a JSON store I'd migrate to Postgres for production." (This turns the 'gotcha' into evidence of range.)

5.5 Python

Basic: list vs tuple vs set vs dict? Mutability? List comprehension? `*args` / `**kwargs` ?

Intermediate: Generators vs lists (memory)? Decorators? Context managers? The GIL — what is it and when does it bite? Virtual environments?

Advanced: multiprocessing vs threading vs asyncio? How would you structure the Smart Glasses inference loop for low latency? Profiling a slow script?

Coding: anagram groups; two-sum; word frequency; flatten nested dict; LRU cache with `OrderedDict` .

5.6 Java & C++ (fundamentals / DSA)

Careful: you list C++ but there's little evidence of it beyond DSA/coursework. If asked "where did you use C++?", answer honestly ("competitive programming and coursework"). Don't imply production C++.

Java basics: OOP pillars (encapsulation/inheritance/polymorphism/abstraction); interface vs abstract class; `==` vs `.equals()` ; ArrayList vs LinkedList vs HashMap; checked vs unchecked exceptions; `final` / `static` .

Advanced: How HashMap works internally (buckets, hashing, collisions, treeify); Java memory model / GC basics; generics; concurrency (`synchronized` , `ExecutorService`).

C++: pointers vs references; RAII; stack vs heap; `vector` internals; move semantics (basic). Keep to what you know from DSA.

5.7 REST APIs, Auth (JWT) & Security

Basic: What is REST? HTTP methods & status codes? Idempotency? What is a JWT and what's inside it?

Intermediate: How does JWT verification work (you verify against Supabase)? Access vs refresh tokens? What is CSRF and how did you guard it? What is CORS? Rate limiting — why per-endpoint?

Advanced: Where do you store tokens (cookie vs localStorage) and the XSS/CSRF trade-off? OWASP

Top 10 — name five and your mitigations? Prompt injection as a new attack surface? Secrets management?

Scenario: "Design auth + authorization for a multi-role app." (Your six-role model + `requireRole` is a ready-made answer.)

5.8 Git/GitHub, Docker, Postman, n8n

Git: merge vs rebase; resolving conflicts; what is a PR; `.gitignore` (and why you should NOT commit `build/` — see your Legal-Ai-App).

Docker: image vs container; what is a Dockerfile; what does docker-compose do (cite TradeLab-AI); why containerise?

n8n: trigger vs action nodes; webhook vs schedule; when NOT to use it.

Postman: how you test/collection-run your APIs.

5.9 Computer Vision & YOLO / Raspberry Pi

Basic: classification vs detection vs segmentation; what YOLO does; bounding boxes/confidence.

Intermediate: IoU & NMS; precision/recall/mAP; why YOLO is single-shot/fast; edge-inference constraints.

Advanced: model quantisation/pruning; fps optimisation on Pi; dataset/annotation; handling class imbalance and false positives in a safety context.

Skills to fix before interviewing: add **Kotlin**, **Android/Jetpack Compose**, **Room**, **Hilt**, **MVVM**, **Prisma**, **Redis** (all code-proven, currently missing). Qualify or make-true **RAG** and **Agentic AI**. Decide whether to keep **C++** given limited evidence.

SECTION 6

Cross Questions — The Skeptical Interviewer

Real interviews are chains, not single questions. Below are full "drill-until-exhausted" ladders on your riskiest claims. Practice saying each rung out loud. The goal: never be surprised by the next "why?".

6.1 Chain: "You claim RAG pipelines."

Q: You list RAG on your resume — walk me through your pipeline. **You:** (Honest) "My current apps use structured LLM prompting; I haven't shipped a full RAG system yet — here's exactly how I'd build one for NyayaSim." **Q:** So it's not actually implemented? **You:** "Correct, and I should tighten the resume wording. But I understand it deeply: chunk the statutes and case law, embed them, store in pgvector on my existing Supabase Postgres, retrieve top-k by cosine similarity, inject with citations." **Q:** Why pgvector, not Pinecone? **You:** "Reuse existing infra, transactional consistency, fine to a few million vectors; I'd move to a dedicated vector DB only past that scale." **Q:** How do you chunk legal text well? **You:** "By logical unit — section/sub-section — with metadata (act, section, year), plus overlap so cross-references survive." **Q:** How do you stop hallucinated citations? **You:** "Instruct the model to answer only from retrieved context, return the cited section IDs, and verify those IDs exist in my corpus before showing them."

Why this wins: you converted an unsupported claim into a demonstration of design depth *and* intellectual honesty. That combination scores higher than a shaky "yes I built it."

6.2 Chain: "Why MongoDB and not PostgreSQL?" (DealHunt)

Q: Why Mongo? **You:** "Document-shaped, evolving deal data; fast to iterate." **Q:** What if you need a transaction across users, deals and payments? **You:** "Mongo supports multi-doc transactions now, but relational integrity is easier in Postgres; if the domain became transaction-heavy I'd move." **Q:** How do you join a user's deals with their votes efficiently in Mongo? **You:** "Embed where read-together, reference where large/independent; use `$lookup` sparingly and index the foreign fields." **Q:** So when is Mongo the wrong choice? **You:** "Heavy cross-entity joins, strict schemas, complex analytics — that's Postgres territory, which I used in Bookstore and TradeLab-AI."

6.3 Chain: "10,000 users log in and use the courtroom simultaneously."

Q: What breaks first? **You:** "The JSON-file datastore — it rewrites the whole file per write and serialises writes; it'll become the bottleneck and can't run across multiple instances." **Q:** Fix it.

You: "Move to Postgres/Prisma (done in TradeLab-AI). Now the API is stateless and scales horizontally behind a load balancer." **Q:** Now what's the next bottleneck? **You:** "LLM latency and rate limits — every turn is a synchronous provider call. I'd add a queue, stream responses (already done for the tutor via SSE), cache generated cases in Redis, and route across my three providers for headroom." **Q:** Cost at that scale? **You:** "Model per-trial token cost, cache aggressively, use the cheapest adequate model (Claude Haiku today), and cap per-user usage." **Q:** How do you know it works before launch? **You:** "Load test with k6/Artillery against a staging deploy, watch p95 latency and error rate, and set SLOs."

6.4 Chain: "Is NyayaSim's AI really 'agentic'?"

Q: Define agentic. **You:** "In my build it means multiple LLM personas — judge, counsel, witnesses — each with its own system prompt and short memory, coordinated by the backend. It's role orchestration." **Q:** That's not autonomous agents though. **You:** "Agreed — there's no plan act observe tool-use loop yet. Calling it 'multi-persona orchestration' is more accurate; true autonomous agents are my next step." **Q:** How would you make it truly agentic? **You:** "Give each persona tools (retrieve statute, look up precedent), a planning loop, and a terminator/guardrail so it can't loop forever, with an orchestrator arbitrating turns."

6.5 Chain: "Prove this is your code."

Q: Explain a specific design decision and why. **You:** Pick the provider-fallback `ask()` or the security middleware ordering — explain the *why*, not just the *what*. **Q:** Why is `requireAuth` after `rateLimit` but before the routes? **You:** "Rate-limit first to cheaply reject floods before doing work; validate JSON and CSRF; then authenticate; then audit — each layer assumes the previous passed." **Q:** What would you refactor? **You:** "Split `claudeService.js` into provider adapters + a router, add a schema validator (Zod) instead of ad-hoc checks, and replace the JSON DB." (Naming real refactors proves ownership.)

6.6 Rapid skeptical volley (practice one-breath answers)

- "Why should I believe your latency numbers?" "Here's how I measured them and the tool I used."
- "What if the LLM is down?" "Provider fallback + offline/guest mode + graceful error normalization."
- "Why not just use one model?" "Vendor risk, cost/latency flexibility, A/B quality."
- "Where's your test suite?" (Honest) "Manual + Postman today; unit tests for services and prompt-contract tests are my next priority." (Don't fake it.)
- "Isn't a JSON file a toy?" "For a demo/offline mode it's deliberate and adequate; for production it's the first thing I'd replace, and I have in TradeLab-AI."

- "How do you know your AI's legal answers are correct?" "I don't fully today — that's why RAG + citation verification + disclaimers matter, and why an eval harness is on the roadmap."
- "What's the one thing you're least confident about here?" Answer sincerely; self-awareness scores.

Meta-technique: when you genuinely don't know, say "I don't know, but here's how I'd find out" and reason aloud. Interviewers cross-question to find your *edge of knowledge* and watch how you behave there — calm honesty beats bluffing every single time.

SECTION 7

Resume Weaknesses & Stronger Rewrites

Every weakness worth fixing, ranked by how much it can hurt you, each with a concrete, evidence-backed rewrite. Only changes supported by your actual code/achievements are suggested.

7.1 Priority Fix List

#	Weakness	Risk	Fix
1	"RAG pipelines" unsupported by any repo	High	Build one real RAG feature OR reword to "LLM prompt-orchestration pipelines."
2	"PostgreSQL" implies NyayaSim backend uses it (it's a JSON file)	High	Attribute Postgres to Bookstore/TradeLab-AI; describe NyayaSim store accurately.
3	Kotlin/Android/Room/Hilt/MVVM missing despite full Android app	Medium (missed upside)	Add them to skills; mention the Android client in NyayaSim's bullet.
4	No metrics anywhere (users, latency, accuracy, time saved)	Medium	Add 1 honest number per project/internship.
5	3 of 4 projects not publicly verifiable; repo naming mismatch	Medium	Publish/clean repos; add GitHub links; rename Legal-Ai-App NyayaSim.
6	"Agentic AI" overclaims vs multi-persona prompting	Medium	Reword to "multi-agent role simulation / LLM orchestration."
7	Docker listed but not in the flagship project	Low	Fine — back it with TradeLab-AI; be ready to cite it.
8	Internship 2 months, no owned outcome/metric	Low	Add a quantified bullet; clarify ownership.
9	GitHub: no READMEs, committed build artifacts	Low	Add READMEs + screenshots + live links; add .gitignore.
10	C++ listed with thin evidence	Low	Keep only if DSA-ready; otherwise drop.

7.2 Bullet Rewrites (before after)

Professional Summary

Before: "...Skilled in LLM applications, RAG pipelines, and agentic AI workflows with the OpenAI and Gemini APIs..."

After (accurate + stronger): "...Skilled in LLM applications and prompt-orchestration pipelines with a provider-agnostic layer over OpenAI, Gemini and Anthropic APIs, plus production-style React/Node platforms with secure auth, layered API security, and a native Kotlin/Android client."
(Every word here is code-backed.)

NyayaSim

Before: "...secure authentication... Designed scalable REST APIs following Clean Architecture and SOLID... React, Node, Express, Supabase, PostgreSQL, OpenAI, Gemini."

After: "Built an AI courtroom (React + Node/Express + Kotlin Android) where LLM personas play judge, counsel and witnesses in English/Hindi, citing Indian statutes. Engineered a provider-agnostic LLM layer (OpenAI/Gemini/Anthropic) with automatic fallback and JSON-schema output parsing. Shipped production-style security: Supabase JWT verification, role-based access (6 roles), CSP/HSTS, CSRF guard, layered rate limiting, upload validation and audit logging. Deployed on Render."

Stack line React, Node.js, Express, Supabase (Auth), OpenAI/Gemini/Anthropic APIs, Kotlin, Jetpack Compose, Room; deployed on Render.

Smart Glasses

Before: "...real-time YOLO object-detection pipeline... optimized for low-latency inference..."

After (add the numbers you actually hit): "Built a real-time YOLO[vN] detection pipeline on Raspberry Pi achieving ~[X] FPS, converting detections to spoken navigation cues via text-to-speech with ~[Y]ms capture-to-audio latency for hands-free assistive navigation." (Fill X/Y with your real measurements.)

Internship

Before: "...eliminate repetitive manual tasks through API integrations."

After: "Built [N] n8n workflows integrating OpenAI/Gemini and a Telegram bot that automated [specific task], cutting a ~[X]-hour manual process to [Y] minutes."

7.3 Buzzwords Without Evidence — verdict

Buzzword	Backed?	Action
Generative AI	Yes	Keep — strongest, most-proven claim.
RAG	No	Build or reword. Do not leave as-is.
Agentic AI	Partial	Reframe as multi-persona orchestration.

Workflow Automation	Yes	Keep — internship + Telegram evidence.
Clean Architecture / SOLID	Partial (strong on Android)	Keep; cite the Android layering as proof.
Scalable REST APIs	Partial	Keep but be ready to concede the JSON-DB limit.

7.4 What's Missing Entirely (add these)

- **Metrics** — even one honest number per item transforms credibility.
- **Testing** — no test strategy mentioned; add "Postman collections; unit tests for services" once true.
- **Deployment detail** — mention Render (NyayaSim) and Docker-Compose (TradeLab-AI).
- **Mobile skills** — Kotlin/Android are entirely absent yet fully built.
- **GitHub links per project** — and fix the NyayaSim Legal-Ai-App naming gap.
- **GATE score/rank** — a number beats "qualified."

Net effect of these fixes: the resume moves from "impressive but slightly over-claimed" (7.4) to "impressive and bullet-proof under code review" (9). None of these changes require inventing anything — they either add real evidence or align words with the code you already wrote.

SECTION 8

Behavioral Interview

STAR = Situation, Task, Action, Result. Answers below are drawn from your real projects and achievements. Personalise the specifics (names, numbers) so they're true and vivid. Aim for 90 seconds each.

B1. Tell me about a challenging technical problem you solved.

S: In NyayaSim, LLM responses that were supposed to be JSON kept arriving wrapped in markdown or prose, crashing my parser and breaking the courtroom flow. **T:** I needed reliable structured output across three different LLM providers. **A:** I wrote strict JSON-only system prompts with schema-shaped examples, lowered temperature, capped tokens, and built a `safeJSON()` helper that strips code fences and regex-extracts the JSON object as a fallback. **R:** Parse failures went from routine to rare, and the same code worked across OpenAI, Gemini and Anthropic. **Reflection:** I later learned provider JSON-mode/function-calling would be even more robust — my next iteration.

B2. Describe a time you failed or made a wrong decision.

S: To ship NyayaSim fast I stored all app data in a single JSON file. **T:** It had to persist performances, posts and audit logs reliably. **A:** It worked for demos, but I realised it couldn't handle concurrent writes or run across multiple instances — a real scalability dead-end. **R:** I documented the limitation, kept it for offline/guest mode, and in my next project (TradeLab-AI) I started with Postgres + Prisma from day one. **Lesson:** match persistence to the reliability requirement early, not after.

B3. Tell me about working under a tight deadline.

S: Hackfinty 2025, team of two, ~24–36 hours to build an AI mental-wellness app. **T:** Ship something demoable, not everything. **A:** We ruthlessly scoped to one core loop, split work (I took [your part]), and integrated early to avoid last-minute merge chaos. **R:** We delivered a working demo on time. **Lesson:** scope-cutting is a skill; a small thing that works beats a big thing that doesn't.

B4. A time you had to learn a new technology quickly.

S: For NyayaSim's mobile client I'd not built native Android before. **T:** Ship a real Kotlin app mirroring the web features. **A:** I learned Jetpack Compose, Hilt DI, and Room, and applied clean architecture (data/domain/feature layers) so the codebase stayed maintainable. **R:** A working native app with offline-seeded cases. **Lesson:** good architecture makes an unfamiliar stack learnable because the structure guides you.

B5. A time you disagreed with someone / handled conflict.

Use a real example. Structure: the disagreement (e.g., tech choice or scope), how you argued with data not ego, how you found common ground or ran a quick experiment to decide, and the outcome. If you lack a strong team example, use a trade-off decision you debated with yourself and justify how you'd handle it on a team.

Painting the other person as an idiot. Show respect + reasoning.

B6. A time you took ownership beyond what was asked.

Ideal: The security middleware in NyayaSim — no one required CSP/CSRF/rate-limiting/audit logging for a student project, but you built production-style hardening because you wanted it to be real. That's ownership.

B7. A time you had to prioritise among many tasks.

Ideal: Building NyayaSim solo across web + backend + mobile forced constant prioritisation. Explain your rule: ship the core courtroom loop first (highest user value), then auth/security, then Academy/extras. Value and dependency order over novelty.

B8. Tell me about difficult debugging.

Ideal: Use the requestId-tracing story (Section 3, Q14): a failing AI call traced via audit-log requestId to a provider timeout, fixed with error normalization and a clearer user message.

B9. A time you received tough feedback.

Ideal: This handbook is a great real example — "a review of my resume against my code showed I'd over-claimed RAG and mislabeled my database. I took it as a chance to align my words with reality and plan the RAG feature properly." Growth mindset in action.

B10. Why do you build side projects at all? / What motivates you?

Ideal: Genuine curiosity + wanting to solve real problems (courtroom practice for students, navigation for the visually impaired). Tie motivation to impact, not just résumé-building.

Behavioral tips: keep a mental "story bank" of 5–6 real incidents; most behavioral questions map onto them. Always end with a **result** and a **lesson**. Use "I" for your actions, "we" for context. Never badmouth anyone.

SECTION 9

HR Interview

Non-technical but decisive — HR screens for fit, motivation, and red flags. Answers tailored to you; adjust specifics before interviewing.

H1. Tell me about yourself (HR version).

Ideal: Shorter, more human than the technical version: "Final-year ECE student at CCET who fell in love with building software, especially AI products. I've built an AI courtroom simulator end-to-end, interned in AI automation, and I keep my fundamentals sharp — I qualified GATE CS and solved 170+ DSA problems. I'm looking for a role where I can grow as an engineer and ship products people actually use."

H2. Why should we hire you?

Ideal: "Three reasons: I build complete products, not just features — NyayaSim spans web, backend, AI and mobile. I have strong fundamentals (GATE CS, DSA rank). And I learn fast and take ownership — I taught myself native Android and shipped production-style security no one asked for. I'll add value quickly and keep growing."

Generic "I'm hardworking and passionate." Give evidence.

H3. Why our company / this role?

Ideal: Research first, then connect: name a specific product/team/value of theirs and link it to your work (e.g., "your AI platform team" "my LLM/full-stack work on NyayaSim"). Show you chose them, not just any offer.

"Because you're a great company / good pay / brand name."

H4. Your strengths?

Ideal: End-to-end product building; fast self-directed learning; fundamentals + delivery balance. Back each with one line of proof.

H5. Your weaknesses?

Ideal (real + improving): "I've sometimes optimised for shipping speed over long-term robustness — like using a quick JSON datastore that wouldn't scale. I've been correcting it by designing the data layer properly upfront, as I did in TradeLab-AI with Postgres/Prisma. Another: I tend to over-claim excitedly — I'm learning to describe things precisely, which is why I'm tightening my resume wording."

Fake weakness ("I'm a perfectionist"). Interviewers see through it.

H6. Where do you see yourself in 3–5 years?

Ideal: Growing from strong individual contributor toward technical depth in AI/backend systems, ideally owning meaningful product areas. Ambitious but grounded; tie to the company's growth paths.

H7. Salary expectations?

Ideal (fresher): "I'm early-career and my priority is learning and impact, so I'm flexible and open to your standard band for this role. If you can share the range, I'm confident we can align." Deflect politely; don't lowball or overshoot blindly. Research market bands beforehand.

H8. Are you considering higher studies?

Careful (you qualified GATE — they may probe M.Tech intent): "I qualified GATE to strengthen my CS fundamentals, and it did. My focus now is industry experience and building real products; I'm committed to this role." (Reassure them you won't leave for a master's in 6 months, if that's true.)

Saying "yes, I'll leave for M.Tech next year" unless it's genuinely your plan.

H9. Relocation / remote / work hours?

Ideal: Be honest and accommodating within your real constraints. "I'm open to relocating to [city]" / "I'm comfortable with your hybrid model." Flexibility helps freshers; don't over-commit to something you can't do.

H10. Notice period / availability?

Ideal: As a student, state your graduation timeline (2027 expected) and earliest full-time availability, plus willingness to intern before then if relevant.

H11. Biggest achievement & biggest regret?

Achievement: GATE CS from an ECE background, or shipping NyayaSim solo across four surfaces.

Regret (light): Not publishing/polishing your GitHub repos earlier — now fixing it. Keep regret professional and recoverable.

H12. Do you have questions for us?

Always yes. Ask 2–3 thoughtful ones: "What does success look like in the first 6 months?", "How does the team use AI/LLMs today?", "What's the biggest technical challenge the team faces?" Never "no questions" — it signals disinterest.

SECTION 10

Coding Interview

Predicted from your profile: you have 170+ DSA problems and GATE CS, so expect solid medium-level DSA plus SQL (you have real schemas) and JS/Node practical rounds. Categorised with difficulty, approach, complexity, and traps.

Your DSA base language is Java (per your DSA repo). Use it unless the interviewer wants JS. Mirror ~50 problems onto LeetCode so you can quote a recognisable count.

10.1 Arrays & Strings

Problem	Diff	Approach	Time / Space
Two Sum	Easy	HashMap of complement	$O(n)$ / $O(n)$
Best Time to Buy/Sell Stock	Easy	Track min + max profit	$O(n)$ / $O(1)$
Longest Substring w/o Repeating	Med	Sliding window + set/map	$O(n)$ / $O(k)$
Group Anagrams	Med	Sorted key or char-count key	$O(nk \log k)$ / $O(nk)$
Product of Array Except Self	Med	Prefix + suffix products	$O(n)$ / $O(1)$
Maximum Subarray (Kadane)	Med	Running max	$O(n)$ / $O(1)$
Merge Intervals	Med	Sort by start, merge overlaps	$O(n \log n)$ / $O(n)$

Common mistakes: off-by-one in sliding windows; forgetting to update the window's left pointer; integer overflow in products (use long).

10.2 HashMaps & Two Pointers

Problem	Diff	Key idea	Time
Valid Anagram	Easy	Count array of 26	$O(n)$
Top K Frequent Elements	Med	Count + heap/bucket sort	$O(n \log k)$
Container With Most Water	Med	Two pointers inward	$O(n)$
3Sum	Med	Sort + two-pointer, skip dups	$O(n^2)$
Longest Consecutive Sequence	Med	Set, start-of-run check	$O(n)$

10.3 Linked Lists, Stacks, Queues

Problem	Diff	Key idea
Reverse Linked List	Easy	Iterative 3-pointer / recursion
Detect Cycle	Easy	Floyd's fast/slow
Merge Two Sorted Lists	Easy	Dummy head + splice
Valid Parentheses	Easy	Stack of open brackets
LRU Cache	Med	HashMap + doubly linked list
Min Stack	Easy	Auxiliary min stack

LRU Cache is high-probability for you — you literally reason about caching in your scaling answers. Be able to code it cold (get/put $O(1)$).

10.4 Trees & Graphs

Problem	Diff	Approach
BFS/DFS traversals, level order	Easy/Med	Queue (BFS) / recursion or stack (DFS)
Max Depth / Diameter of Tree	Easy/Med	Post-order recursion
Validate BST	Med	In-order or min/max bounds
Lowest Common Ancestor	Med	Recursive split search
Number of Islands	Med	Grid DFS/BFS flood fill
Course Schedule (cycle detect)	Med	Topological sort / DFS colors
Clone Graph	Med	DFS/BFS + visited map

10.5 Dynamic Programming

Problem	Diff	Recurrence idea
Climbing Stairs / Fibonacci	Easy	$dp[i] = dp[i-1] + dp[i-2]$
House Robber	Med	$dp[i] = \max(dp[i-1], dp[i-2] + a[i])$
Coin Change	Med	Min coins over amounts
Longest Common Subsequence	Med	2D grid on two strings
0/1 Knapsack	Med	Weight-capacity table

Traps: jumping to code before writing the recurrence + base cases; not stating the state definition aloud; missing memoisation exponential blow-up.

10.6 SQL

You have real schemas (Bookstore, TradeLab-AI) — expect SQL. Practice against your own Bookstore tables (Books/Customers/Orders):

1. Top 5 customers by total order amount (JOIN + GROUP BY + ORDER BY + LIMIT).
2. Books never ordered (LEFT JOIN ... WHERE Order_ID IS NULL, or NOT IN/NOT EXISTS).
3. 2nd highest book price (subquery / `DENSE_RANK()`).
4. Monthly revenue (GROUP BY date_trunc).
5. Running total of revenue (window function `SUM() OVER(ORDER BY ...)`).
6. Genre with highest average price (GROUP BY + HAVING/ORDER).
7. Find duplicate customer emails.

Know cold: INNER vs LEFT JOIN, GROUP BY vs WHERE vs HAVING, subqueries vs CTEs, window functions, indexes and when they help.

10.7 OOP & Language

Design a parking lot / library / elevator (OOP modelling with classes, inheritance, interfaces). Explain SOLID with an example from your code. Java specifics: HashMap internals, equals/hashCode contract, immutability, exceptions. JS specifics: closures, prototypes, event loop, `this` binding.

10.8 Backend / API & Practical

Given your profile, expect a practical build/extend round more than pure algorithms:

1. Implement a rate limiter (fixed window / sliding window / token bucket) — you already use one, so explain the algorithm too.
2. Design REST endpoints for a resource (proper verbs, status codes, pagination, validation).
3. Implement retry-with-exponential-backoff for a flaky API call (your LLM calls).
4. Write middleware that authenticates a JWT and attaches the user (you've done this).
5. Debounce/throttle in JS; implement `Promise.all` ; async queue with concurrency limit.
6. Design a URL shortener / paste bin (classic system-design-lite).

10.9 Interview Tips (coding round)

- Clarify inputs/outputs/constraints **before** coding. Ask about edge cases (empty, nulls, duplicates, huge N).
- State your approach and its complexity **before** writing — get buy-in.
- Think out loud; a silent screen is the #1 way to fail even with the right answer.
- Start with brute force, then optimise — don't freeze chasing the optimal first.
- Test your code on a small example and an edge case at the end.
- Know time/space complexity of everything you write.

Company-Specific Preparation

Where each company focuses, and the resume-based questions they're most likely to ask *you*. Prioritise by where you're actually applying.

Company	Interview focus	Likely question for YOU
Google	DSA depth, algorithmic rigour, "Googleness", data structures	Hard graph/DP problem; "design the RAG retrieval at scale"; clean complexity analysis. Your GATE CS + DSA rank is the door here.
Amazon	Leadership Principles (behavioral is ~50%) + DSA	Expect 4–6 LP questions — Ownership, Dive Deep, Bias for Action, Deliver Results. Map your NyayaSim/hackathon/JSON-DB stories to LPs (Section 8). "Tell me about a time you dived deep to debug."
Microsoft	Balanced DSA + design + collaboration, practical coding	Trees/strings; "design a courtroom-like real-time system"; how you work in a team. Friendly, thorough.
Adobe	DSA + CS fundamentals (OS/DBMS/OOP) + project depth	Strong on fundamentals — your GATE prep shines. Deep project grill on NyayaSim architecture + a medium DSA.
Atlassian	Practical full-stack, values-based behavioral, real-world coding	Extend a small app/API live; "design an API for X"; teamwork values. Your full-stack + REST experience fits well.
Oracle / Salesforce	DBMS/SQL depth, backend, OOP	SQL joins/normalisation (use Bookstore), transactions, Java OOP. Cite your Postgres/Prisma work.
NVIDIA / Qualcomm	Systems, C++, embedded, CV/ML close to hardware	Smart Glasses is your hook — edge inference, YOLO optimisation, C/C++, latency. Be ready with real FPS numbers.
Samsung	C/C++, DSA, embedded + some ML	Embedded + Smart Glasses + DSA. Coding in C/C++.
Walmart / Flipkart / Meesho	DSA (strong) + scalable system design	Medium-hard DSA + "design a deal/e-commerce feed" — DealHunt is directly relevant. Scale + caching questions.
PhonePe / Razorpay / CRED	DSA + backend depth + reliability/consistency, high bar	Concurrency, idempotency, transactions, rate limiting (you built one!), security. Fintech loves your security layer.
Zoho / Freshworks	Fundamentals, practical coding, product thinking, DBMS	Multiple coding + aptitude rounds; solid fundamentals and clear projects. Less brand-name DSA grind, more breadth.

Swiggy / Zomato	DSA + real-time systems, high scale	"Design real-time order/feed"; caching, queues; medium-hard DSA. Map to your scaling answers.
Startups	Ship-fast full-stack, ownership, breadth, AI product sense	Your sweet spot — end-to-end building, AI integration, learning speed. Expect "build this feature by tomorrow" energy and a take-home.

11.1 How to re-weight your prep by target

- **Product giants (Google/Amazon/Microsoft/Adobe):** DSA is the gate — grind LeetCode mediums, lean on GATE fundamentals, and prep Amazon LPs hard.
- **Fintech (PhonePe/Razorpay/CRED):** emphasise your security middleware, rate limiting, auth, and reliability thinking — you have real evidence most freshers lack.
- **Hardware/systems (NVIDIA/Qualcomm/Samsung):** make Smart Glasses bullet-proof with real numbers, brush up C++ and CV fundamentals.
- **Startups & AI-first companies:** lead with NyayaSim end-to-end, the provider-agnostic LLM layer, and shipping speed. This is where your profile is strongest right now.

Universal: every company that opens your GitHub will see the JSON-DB, the Legal-Ai-App naming, and the missing RAG. Fix or pre-empt those (Sections 2 & 7) before applying anywhere.

Final Revision Guide

Concise, high-yield lists for last-mile revision. Stars mark probability given *your* resume. If you only have an hour, revise the items.

12.1 Top 30 Highest-Probability Resume Questions

1. Tell me about yourself.
2. Walk me through NyayaSim end-to-end.
3. Why did you support three LLM providers? Which is deployed?
4. Your resume says OpenAI/Gemini but you deploy Claude — explain.
5. Explain your RAG pipeline.
6. Where's your PostgreSQL code?
7. How do you get structured JSON out of an LLM reliably?
8. How does your authentication work?
9. Biggest scalability bottleneck in NyayaSim?
10. Is your AI really "agentic"? Define it.
11. Why Node/Express over Python?
12. Security risks of sending user text to an LLM?
13. 10,000 concurrent users — what breaks?
14. Solo or team on NyayaSim — what did YOU build?
15. Why an ECE student in a software role?
16. What's your GATE CS score/rank?
17. Biggest technical challenge you solved?
18. Tell me about a failure.
19. Why MongoDB for DealHunt?
20. How exactly are DealHunt's recommendations "AI-powered"?
21. Smart Glasses: which YOLO, what FPS?
22. How is the internship Telegram bot different from your Telegram project?
23. What is prompt engineering — give a real technique.
24. Why is your repo "Legal-Ai-App" but resume says "NyayaSim"?
25. Where's your test suite?
26. What would you refactor in NyayaSim?
27. How did you deploy it?
28. Explain your security middleware chain.
29. What metric proves your project is good?
30. Do you have questions for us?

12.2 Top 20 Project Questions

1. Narrate one courtroom turn's full request lifecycle.
2. How does provider fallback work + its weaknesses?
3. How do you verify the Supabase token server-side?

4. Your six roles and how authorization is enforced.
5. SSE streaming for the tutor — why SSE over WebSockets?
6. Migrate the JSON store to Postgres with zero downtime.
7. Defend against prompt injection in witness/lawyer prompts.
8. Cache generated cases — what's the key, how avoid staleness?
9. Retry + circuit breaker for provider failover.
10. Eval harness for legal-answer quality.
11. The Android clean-architecture layering.
12. Why Room + offline seeding on mobile?
13. Cost model per trial + three levers to cut it 10x.
14. Design the full RAG pipeline (pgvector).
15. How do you stop hallucinated case citations?
16. Rate-limiting fairness per-user behind a proxy.
17. Where could XSS enter and how to close it?
18. Smart Glasses: box speech + latency budget.
19. DealHunt data model + indexes.
20. TradeLab-AI: Redis usage + docker-compose services.

12.3 Top 20 Technical / Fundamentals Questions

1. Explain the Node.js event loop.
2. Node single-threaded — handling concurrency.
3. SQL vs NoSQL — when each.
4. INNER vs LEFT JOIN; GROUP BY vs HAVING.
5. What is a JWT; access vs refresh tokens.
6. CORS vs CSRF vs XSS — differences + defenses.
7. REST principles + idempotency + status codes.
8. React re-render triggers; useMemo/useCallback.
9. useEffect dependency-array pitfalls.
10. HashMap internals (buckets/collisions/treeify).
11. ACID + transaction isolation levels.
12. Indexing — benefit and cost.
13. Process vs thread; Python GIL.
14. OOP four pillars + a SOLID example from your code.
15. What is a token / temperature / context window.
16. RAG components end-to-end.
17. Caching strategies + eviction (LRU/TTL).
18. Docker image vs container; compose.

19. Git merge vs rebase; .gitignore hygiene.
20. YOLO: IoU, NMS, mAP.

12.4 Top HR & Behavioral (quick list)

HR (Section 9)	Behavioral (Section 8)
Tell me about yourself	Biggest technical challenge
Why should we hire you?	A time you failed
Why our company/role?	Learned a tech fast (Android)
Strengths / Weaknesses	Tight deadline (Hackfinitly)
Higher studies intent?	Took ownership (security layer)
Salary expectations	Difficult debugging (requestId)
3–5 year goals	Conflict / disagreement
Questions for us	Prioritising many tasks

12.5 Top System-Design & Debugging Prompts

System design: Scale NyayaSim to 1M users · Design a RAG service · Design a URL shortener · Design a deal/news feed (DealHunt/Telegram) · Design a rate limiter · Design real-time multiplayer courtroom · Multi-region LLM failover + cost routing .

Debugging: Trace a failing AI call by requestId · Diagnose a 3x cost spike · Find a React re-render/perf bug · Reproduce an intermittent JSON-parse failure · Debug a provider timeout/rate-limit · Memory leak in a long-running Node process .

Night-Before Interview Cheat Sheet

One-page-per-topic revision. Read this last thing tonight and first thing tomorrow. Breathe. You've built real things — this just helps you talk about them clearly.

A. Your 30-Second Self-Pitch (say it out loud 3×)

"I'm Rahul, a final-year ECE student at CCET working in Generative AI and full-stack. My flagship is NyayaSim — an AI courtroom with a React front end, a Node backend, a provider-agnostic LLM layer over OpenAI/Gemini/Claude, and a native Kotlin Android app. I interned in AI automation, qualified GATE CS, and solved 170+ DSA problems. I build AI products end to end."

B. NyayaSim in One Breath (your anchor project)

Stack: React+Vite · Node/Express (ESM) · Supabase auth · JSON datastore · OpenAI/Gemini/Anthropic · Kotlin/Compose Android · Render.

Flow: client securityHeaders rateLimit validateJson csrfGuard requireAuth auditLog route service ask() LLM safeJSON response.

Deployed model: claude-haiku-4-5 . **Roles:** student/lawyer/judge/admin/college/recruiter.

AI fns: generateCase, judgeRespond, lawyerRespond, witnessAnswer, objection, analyzePerformance, deliverJudgment, getPrecedents, tutor(SSE).

C. The Three Honest Scripts (rehearse verbatim)

RAG: "I use structured LLM prompting today; full RAG (pgvector over the legal corpus, retrieve-then-cite) is my next step — here's the design."

PostgreSQL: "NyayaSim uses Supabase (Postgres) for auth; app data is a JSON store for demos/offline. Real Postgres is in my Bookstore and TradeLab-AI (Prisma) projects."

Agentic: "Multi-persona LLM orchestration with per-role prompts and memory — not autonomous tool-using agents yet; that's the roadmap."

D. Key Definitions (10-second answers)

RAG	Retrieve relevant docs (embed vector search) then feed to the LLM to answer with grounded citations.
JWT	Signed token (header.payload.signature) proving identity; verified server-side, no session storage needed.

REST	Stateless HTTP API using nouns + verbs (GET/POST/PUT/DELETE) and status codes.
Event loop	Node's single-threaded scheduler that handles many I/O ops concurrently via callbacks/promises.
CSRF	Forcing a logged-in user's browser to make unwanted requests; defended by tokens (double-submit).
ACID	Atomicity, Consistency, Isolation, Durability — DB transaction guarantees.
Idempotency	Same request repeated = same result (safe retries).
NMS (YOLO)	Non-max suppression: keep the best overlapping box, drop the rest by IoU.
Temperature	LLM randomness knob; low = deterministic, high = creative.
Index	DB structure that speeds reads at the cost of slower writes + storage.

E. Complexity Cheat Table

Structure/Op	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
HashMap	—	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
Balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap	—	$O(n)$	$O(\log n)$	$O(\log n)$ (min/max)
Sorting (merge/heap)	$O(n \log n)$ time			
BFS/DFS graph	$O(V + E)$			

F. Architecture Sketch to Redraw on the Whiteboard

```

[React SPA] [Kotlin Android]
  \       /
  \   /   HTTPS + JWT
  [ Express API ]
headers limit json csrf auth audit
  |           |
[ ask() LLM ] [ datastore ]
OpenAI/Gemini JSON now  Postgres next
/Anthropic  (+ Redis cache, queue @ scale)
  |
[ (future) RAG: embed  pgvector  retrieve  cite ]

```

G. Common Pitfalls — Don't Do These Tomorrow

- Don't claim RAG/Postgres/Agentic without the honesty scripts — you'll get caught.
- Don't say "we built" then fail "which part was yours?" — know your exact contribution.
- Don't code silently — narrate. Don't skip clarifying questions.
- Don't badmouth anyone in behavioral answers.
- Don't say "no questions for you."
- Don't fake a test suite or metrics — say "manual/Postman today, here's my plan."
- Don't forget: your deployed model is Claude Haiku, your repo is Legal-Ai-App.

H. Bring / Have Ready

- GATE scorecard, GeeksforGeeks profile (170+, rank 16), GitHub open to Legal-Ai-App + TradeLab-AI.
- A 2-line answer for each of the four projects + which are on GitHub.
- Your "story bank" of 5–6 STAR incidents.
- 2–3 questions to ask them, researched about the specific team.
- Water, charged laptop, quiet space, and the self-pitch rehearsed.

I. The Top 30 Highest-Probability Questions (from 12.1) — final skim

Re-read Section 12.1 now. If you can answer #1–#15 crisply and deliver the three honesty scripts (C) without flinching, you are ready. Then sleep — a rested brain out-interviews a crammed one.

Final word. Your resume is genuinely strong and most of it is backed by real, well-engineered code — that already puts you ahead. The only thing standing between you and a great interview is being the person who explains the gaps first. Do that, stay honest, reason out loud, and let the work speak. Good luck, Rahul.